

ASSESSMENT TOOL 1-DATABASE DESIGN AND CONNECTIVITY

Course Code: DSA0504

**Course Title: Query
Processing for Data
Science**

CO Assessed: CO2

**Assessment: Tool 1 –
Database design and
connectivity**

Weightage: 40

Total Marks: 25 Marks

CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)

Student Name: Jeya Harshini R

Reg. No.:192424051

Section / Batch:2024

**Date of
Submission:3/8/26**

**Faculty In-charge: Dr. R.Mahesh
Muthulakshmi**

**Project Mode: Individual
Submission**

Question 1: Student Database Design and Connectivity

Question

A college wants to automate the management of student records. Design a PostgreSQL database containing a **Student** table with appropriate fields and a primary key. Write a Python program to connect to the PostgreSQL database, create the table if it does not exist, insert at least five student records, and display all the records stored in the table.

Objective

- To establish connectivity between Python and PostgreSQL.
 - To create a Student table using SQL.
 - To insert multiple student records into the database.
 - To retrieve and display all records stored in the table.
-

Database Connectivity

Database connectivity is the process of connecting an application to a database management system to perform operations such as creating tables, inserting records, updating data, deleting records, and retrieving information. In this experiment, Python establishes a connection with the PostgreSQL database using the **psycopg2** library. The `connect()` method creates the database connection, while the cursor object executes SQL commands. After executing SQL queries, `commit()` permanently saves the changes, and `close()` safely terminates the connection.

Algorithm

1. Import the `psycopg2` library.
2. Connect to the PostgreSQL database.
3. Create the Student table if it does not exist.
4. Insert five student records.
5. Commit the transaction.
6. Retrieve all student records.
7. Display the records.

8. Close the cursor and database connection.

Python Program

```
student.py - C:/Users/Harshini Darshan/OneDrive/Documents/student.py (3.11.9)
File Edit Format Run Options Window Help

import psycopg2

conn = psycopg2.connect(
    host="localhost",
    database="college",
    user="postgres",
    password="jeyah812",
    port="5432"
)

cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS Student(
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Department VARCHAR(50),
    Age INT,
    Marks FLOAT
)
""")

cur.execute("DELETE FROM Student")

students = [
    (101, "Alice", "AI", 20, 91.5),
    (102, "Bob", "CSE", 21, 85.0),
    (103, "Charlie", "ECE", 19, 88.5),
    (104, "David", "IT", 20, 79.0),
    (105, "Eva", "AI", 21, 95.2)
]

cur.executemany("INSERT INTO Student VALUES (%s,%s,%s,%s,%s)", students)

conn.commit()

cur.execute("SELECT * FROM Student")

print("Student Records")
for row in cur.fetchall():
    print(row)

cur.close()
conn.close()
```

Output

```
Student Records
(101, 'Alice', 'AI', 20, 91.5)
(102, 'Bob', 'CSE', 21, 85.0)
(103, 'Charlie', 'ECE', 19, 88.5)
(104, 'David', 'IT', 20, 79.0)
(105, 'Eva', 'AI', 21, 95.2)
```

Description

In this experiment, Python was successfully connected to the PostgreSQL database using the psycopg2 library. A Student table was created with appropriate attributes and a primary key. Five student records were inserted into the table using SQL INSERT statements. Finally, the records were retrieved using the SELECT statement and displayed successfully, demonstrating Python–PostgreSQL database connectivity.

Result

The PostgreSQL database was successfully connected with Python. The Student table was created, records were inserted successfully, and all student details were retrieved and displayed.

Question 2: Employee and Department Database Design

Question

A company wants to store employee information and department details in a PostgreSQL database. Design two related tables, **Department** and **Employee**, by identifying appropriate primary and foreign keys. Write a Python program to connect to the PostgreSQL database, create both tables, insert sample data into each table, and display the employee name along with the corresponding department name using an SQL **JOIN** query.

Objective

- To create two related tables using primary and foreign keys.
 - To establish connectivity between Python and PostgreSQL.
 - To perform SQL JOIN operations.
 - To display employee names along with their department names.
-

Database Connectivity

Python communicates with PostgreSQL through the psycopg2 library. Two tables, Department and Employee, are created, where the Department table stores department details and the Employee table stores employee information with a foreign key reference. SQL JOIN combines data from both tables to display meaningful information.

Algorithm

1. Import psycopg2.
2. Connect to PostgreSQL.
3. Create Department table.
4. Create Employee table.
5. Insert department records.
6. Insert employee records.

7. Execute INNER JOIN.
8. Display employee and department names.
9. Close the connection.

Python Program

```
student.py - C:/Users/Harshini Darshan/OneDrive/Documents/student.py (3.11.9)
File Edit Format Run Options Window Help
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    database="college",
    user="postgres",
    password="jeyah812",
    port="5432"
)

cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS Department(
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(50)
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS Employee(
    Emp_ID INT PRIMARY KEY,
    Emp_Name VARCHAR(50),
    Salary NUMERIC,
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
)
""")

cur.execute("DELETE FROM Employee")
cur.execute("DELETE FROM Department")

departments = [
    (1,"HR"),
    (2,"Finance"),
    (3,"IT")
]

employees = [
    (101,"John",45000,1),
    (102,"Mary",55000,2),
    (103,"Steve",60000,3),
    (104,"Anita",50000,3)
]

cur.executemany("INSERT INTO Department VALUES (%s,%s)", departments)
cur.executemany("INSERT INTO Employee VALUES (%s,%s,%s,%s)", employees)

conn.commit()
```

Output

```
=====
Employee Name    Department
('John', 'HR')
('Mary', 'Finance')
('Steve', 'IT')
('Anita', 'IT')
```

Description

The experiment demonstrates the use of relational databases by creating Department and Employee tables. The Employee table references the Department table using a foreign key. An SQL INNER JOIN retrieves employee names along with their corresponding department names, illustrating the relationship between tables.

Result

Python successfully connected to PostgreSQL, created related tables, inserted records, and displayed the joined data using an SQL JOIN query.

Question 3: Inventory Management System

Question

A retail store needs to maintain product inventory using a PostgreSQL database. Design a Product table containing Product ID, Product Name, Category, Quantity, and Unit Price. Write a Python program to connect to the PostgreSQL database, create the table, insert sample product records, identify products with a quantity less than 10, update their stock quantity, and display the updated inventory.

Objective

- To connect Python with PostgreSQL.
 - To create a Product table.
 - To insert inventory records.
 - To identify products with low stock.
 - To update product quantities.
-

Database Connectivity

The PostgreSQL database is connected using the psycopg2 library. SQL commands are executed through the cursor object. Product information is stored in the Product table, and UPDATE statements modify stock quantities based on inventory conditions.

Algorithm

1. Connect to PostgreSQL.
2. Create Product table.
3. Insert product records.
4. Retrieve products with quantity less than 10.
5. Update stock quantity.

6. Display updated inventory.
7. Close the database connection.

Python Program

```
student.py - C:/Users/Harshini Darshan/OneDrive/Documents/student.py (3.11.9)
File Edit Format Run Options Window Help

import psycopg2

conn = psycopg2.connect(
    host="localhost",
    database="college",
    user="postgres",
    password="jeyah812",
    port="5432"
)

cur = conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS Product(
    Product_ID INT PRIMARY KEY,
    Product_Name VARCHAR(50),
    Category VARCHAR(50),
    Quantity INT,
    Unit_Price NUMERIC
)
""")

cur.execute("DELETE FROM Product")

products = [
    (1, "Laptop", "Electronics", 5, 60000),
    (2, "Mouse", "Electronics", 25, 500),
    (3, "Keyboard", "Electronics", 8, 1200),
    (4, "Chair", "Furniture", 12, 2500),
    (5, "Table", "Furniture", 6, 4500)
]

cur.executemany("INSERT INTO Product VALUES (%s,%s,%s,%s,%s)", products)

conn.commit()

print("Products with Quantity < 10")

cur.execute("SELECT * FROM Product WHERE Quantity<10")

for row in cur.fetchall():
    print(row)

cur.execute("""
UPDATE Product
SET Quantity = Quantity + 20
WHERE Quantity < 10
""")

conn.commit()
```

Output

```
Products with Quantity < 10
(1, 'Laptop', 'Electronics', 5, Decimal('60000'))
(3, 'Keyboard', 'Electronics', 8, Decimal('1200'))
(5, 'Table', 'Furniture', 6, Decimal('4500'))

Updated Inventory
(2, 'Mouse', 'Electronics', 25, Decimal('500'))
(4, 'Chair', 'Furniture', 12, Decimal('2500'))
(1, 'Laptop', 'Electronics', 25, Decimal('60000'))
(3, 'Keyboard', 'Electronics', 28, Decimal('1200'))
(5, 'Table', 'Furniture', 26, Decimal('4500'))
```

Description

The Inventory Management System stores product details in a PostgreSQL database. Products with stock quantities below 10 are identified using a SELECT query. Their stock levels are updated using an UPDATE statement, and the revised inventory is displayed. This demonstrates database connectivity, record retrieval, and modification using Python.

Result

The Product table was successfully created in PostgreSQL. Inventory records were inserted, low-stock products were identified and updated, and the final inventory was displayed successfully.
